ЛАБОРАТОРНА РОБОТА № 2

Тема: Організація коду в мові Go. Пакети. Тестування

Мета: Розглянути організацію коду в мові Go: головнийпакет;додаткові пакети, зовнішні пакети. Область видимості ідентифікаторів пакета. Отримати практичні навички у створенні пакетів (одного, двохі більше),організувати тестування функцій пакета.

хід роботи

Завдання 1.

Реалізувати програму в одному вхідному файлі з трьомафункціями: знаходження мінімального значення з трьох елементів, обчислення середнього-значення трьох елементів і рішення рівняння першого порядку;

Виконання завдання:

Код програми:

```
package main

import (
    "errors"
    "fmt"
)

// Min3 повертає мінімальне значення з трьох елементів
func Min3(a, b, c float64) float64 {
    min := a
    if b < min {
      min = b
    }
    if c < min {
      min = c
    }
    return min
```

```
}

// Average3 обчислює середнє значення трьох елементів
func Average3(a, b, c float64) float64 {
    return (a + b + c) / 3
}

// SolveLinear розв'язує рівняння першого порядку ax + b = 0
// Повертає x = -b/a або помилку, якщо a = 0
func SolveLinear(a, b float64) (float64, error) {
    if a == 0 {
        if b == 0 {
            return 0, errors.New("рівняння 0*x + 0 = 0 має нескінченно багато розв'язків")
        }
        return 0, errors.New("рівняння 0*x + b = 0 не має розв'язку")
    }
    return -b / a, nil
}

func main() {
    x, y, z := 5.0, 2.0, 8.0

    fmt.Printf("Числа: %.2f, %.2f, %.2f\n", x, y, z)
    fmt.Printf("Мінімальне значення: %.2f\n", Min3(x, y, z))
    fmt.Printf("Середнє значення: %.2f\n", Average3(x, y, z))

    a, b := 2.0, 4.0
    solution, err := SolveLinear(a, b)
    if err != nil {
        fmt.Printf("Рівняння %.2f*x + %.2f = 0: %v\n", a, b, err)
    } else {
        fmt.Printf("Рівняння %.2f*x + %.2f = 0 => x = %.2f\n", a, b, solution)
    }

    solution2, err2 := SolveLinear(0, 5)
```

```
    if err2 != nil {
        fmt.Printf("Рівняння 0*x + 5 = 0: %v\n", err2)
    } else {
        fmt.Printf("x = %.2f\n", solution2)
    }
}
```

Результат виконання:

```
Числа: 5.00, 2.00, 8.00
Мінімальне значення: 2.00
Середнє значення: 5.00
Рівняння 2.00*x + 4.00 = 0 => x = -2.00
Рівняння 0*x + 5 = 0: рівняння 0*x + b = 0 не має розв'язку

Process finished with the exit code 0
```

Рис. 2.1

Завдання 2.

Розбити додаток з 2 вихідними файлами (у другійфайлвинестиреалізовані три функції);

Виконання завдання:

Код програми:

Файл funcs.go

```
package main

import "errors"

// Min3 повертає мінімальне значення з трьох елементів
func Min3(a, b, c float64) float64 {
    min := a
    if b < min {
        min = b
    }
    if c < min {
        min = c
    }
```

```
    return min
}

// Average3 обчислює середнє значення трьох елементів
func Average3(a, b, c float64) float64 {
    return (a + b + c) / 3
}

// SolveLinear розв'язує рівняння першого порядку ax + b = 0
func SolveLinear(a, b float64) (float64, error) {
    if a == 0 {
        if b == 0 {
            return 0, errors.New("рівняння має нескінченно багато розв'язків")
        }
        return 0, errors.New("рівняння не має розв'язку")
    }
    return -b / a, nil
}
```

Файл main.go

```
package main

import "fmt"

func main() {
    x, y, z := 5.0, 2.0, 8.0

    fmt.Printf("Числа: %.2f, %.2f, %.2f\n", x, y, z)
    fmt.Printf("Мінімальне значення: %.2f\n", Min3(x, y, z))
    fmt.Printf("Середнє значення: %.2f\n", Average3(x, y, z))

    coefA, coefB := 2.0, 4.0
    solution, err := SolveLinear(coefA, coefB)
    if err != nil {
        fmt.Printf("Рівняння %.2f*x + %.2f = 0: %v\n", coefA, coefB, err)
    } else {
```

```
    fmt.Printf("Рівняння %.2f*x + %.2f = 0 => x = %.2f\n", coefA, coefB, solution)
  }
}
```

Результат виконання:

```
Числа: 5.00, 2.00, 8.00
Мінімальне значення: 2.00
Середнє значення: 5.00
Рівняння 2.00*x + 4.00 = 0 => x = -2.00

Process finished with the exit code 0
```

Рис. 2.2

Завдання 3.

Реалізувати пакет (в пакет винести реалізовані три функції);

Виконання завдання:

Код програми:

Файл calc/calc.go:

```
package calc


import "errors"


// Min3 повертає мінімальне значення з трьох елементів
func Min3(a, b, c float64) float64 {
  min := a
  if b < min {
    min = b
  }
  if c < min {
    min = c
  }
  return min
}


// Average3 обчислює середнє значення трьох елементів
func Average3(a, b, c float64) float64 {
  return (a + b + c) / 3
```

```
}

// SolveLinear розв'язує рівняння першого порядку ax + b = 0
func SolveLinear(a, b float64) (float64, error) {
    if a == 0 {
        if b == 0 {
            return 0, errors.New("рівняння має нескінченно багато розв'язків")
        }
        return 0, errors.New("рівняння не має розв'язку")
    }
    return -b / a, nil
}
```

Файл main.go:

```
package main

import (
    "fmt"

    "lab2ex3/calc"
)

func main() {
    x, y, z := 5.0, 2.0, 8.0

    fmt.Printf("Числа: %.2f, %.2f, %.2f\n", x, y, z)
    fmt.Printf("Мінімальне значення: %.2f\n", calc.Min3(x, y, z))
    fmt.Printf("Середнє значення: %.2f\n", calc.Average3(x, y, z))

    coefA, coefB := 2.0, 4.0
    solution, err := calc.SolveLinear(coefA, coefB)
    if err != nil {
        fmt.Printf("Рівняння %.2f*x + %.2f = 0: %v\n", coefA, coefB, err)
    } else {
        fmt.Printf("Рівняння %.2f*x + %.2f = 0 => x = %.2f\n", coefA, coefB, solution)
```

```
    }
}
```

Результат виконання:

```
Числа: 5.00, 2.00, 8.00
Мінімальне значення: 2.00
Середнє значення: 5.00
Рівняння 2.00*x + 4.00 = 0 => x = -2.00

Process finished with the exit code 0
```

Рис. 2.3

Завдання 4.

1. Додайте тести для вашого пакета;

Виконання завдання:

Код програми:

Файл calc.go

```
package calc


import "errors"


// Min3 повертає мінімальне значення з трьох елементів
func Min3(a, b, c float64) float64 {
    min := a
    if b < min {
        min = b
    }
    if c < min {
        min = c
    }
    return min
}


// Average3 обчислює середнє значення трьох елементів
func Average3(a, b, c float64) float64 {
    return (a + b + c) / 3
}
```

```
// SolveLinear розв'язує рівняння першого порядку ax + b = 0
func SolveLinear(a, b float64) (float64, error) {
    if a == 0 {
        if b == 0 {
            return 0, errors.New("рівняння має нескінченно багато розв'язків")
        }
        return 0, errors.New("рівняння не має розв'язку")
    }
    return -b / a, nil
}
```

Файл calc_test.go

```
package calc

import "testing"

func TestMin3(t *testing.T) {
    tests := []struct {
        a, b, c float64
        want   float64
    }{
        {5, 2, 8, 2},
        {-1, -5, -3, -5},
        {0, 0, 0, 0},
        {3, 3, 3, 3},
    }
    for _, tt := range tests {
        got := Min3(tt.a, tt.b, tt.c)
        if got != tt.want {
            t.Errorf("Min3(%v, %v, %v) = %v, потрібно %v", tt.a, tt.b, tt.c, got, tt.want)
        }
    }
}

func TestAverage3(t *testing.T) {
    tests := []struct {
```

```
        a, b, c float64
        want   float64
    }{
        {1, 2, 3, 2},
        {0, 0, 0, 0},
        {3, 6, 9, 6},
    }
    for _, tt := range tests {
        got := Average3(tt.a, tt.b, tt.c)
        if got != tt.want {
            t.Errorf("Average3(%v, %v, %v) = %v, потрібно %v", tt.a, tt.b, tt.c, got, tt.want)
        }
    }
}

func TestSolveLinear(t *testing.T) {
    x, err := SolveLinear(2, 4)
    if err != nil {
        t.Fatalf("SolveLinear(2, 4) помилка: %v", err)
    }
    if x != -2 {
        t.Errorf("SolveLinear(2, 4) = %v, потрібно -2", x)
    }

    x2, err2 := SolveLinear(-3, 6)
    if err2 != nil {
        t.Fatalf("SolveLinear(-3, 6) помилка: %v", err2)
    }
    if x2 != 2 {
        t.Errorf("SolveLinear(-3, 6) = %v, потрібно 2", x2)
    }

    _, err3 := SolveLinear(0, 5)
    if err3 == nil {
        t.Error("SolveLinear(0, 5) повинен повертати помилку")
```

```
    }

    _, err4 := SolveLinear(0, 0)
    if err4 == nil {
        t.Error("SolveLinear(0, 0) повинен повертати помилку")
    }
}

func BenchmarkMin3(b *testing.B) {
    for i := 0; i < b.N; i++ {
        Min3(5, 2, 8)
    }
}

func BenchmarkAverage3(b *testing.B) {
    for i := 0; i < b.N; i++ {
        Average3(1, 2, 3)
    }
}

func BenchmarkSolveLinear(b *testing.B) {
    for i := 0; i < b.N; i++ {
        SolveLinear(2, 4)
    }
}
```

Файл main.go

```
package main

import (
    "fmt"

    "lab2ex4/calc"
)
```

```
func main() {
   x, y, z := 5.0, 2.0, 8.0

   fmt.Printf("Числа: %.2f, %.2f, %.2f\n", x, y, z)
   fmt.Printf("Мінімальне значення: %.2f\n", calc.Min3(x, y, z))
   fmt.Printf("Середнє значення: %.2f\n", calc.Average3(x, y, z))

   coefA, coefB := 2.0, 4.0
   solution, err := calc.SolveLinear(coefA, coefB)
   if err != nil {
      fmt.Printf("Рівняння %.2f*x + %.2f = 0: %v\n", coefA, coefB, err)
   } else {
      fmt.Printf("Рівняння %.2f*x + %.2f = 0 => x = %.2f\n", coefA, coefB, solution)
   }
}
```

Висновок

У результаті виконання лабораторної роботи було опановано повний цикл розробки консольного додатка на мові **Go**: від написання алгоритму розв’язання лінійних рівнянь до організації модульної структури проєкту.

Контрольні питання

1. Як організована структура коду в мові Go?

Код у Go базується на **пакетній системі**. Кожен файл із вихідним кодом належить до певного пакета (що зазначається директивою package на початку файлу). Пакети дозволяють групувати логічно пов'язаний функціонал, забезпечують ізоляцію імен та спрощують повторне використання коду.

2. Що таке головний пакет?

Package main — це спеціальний пакет, який повідомляє компілятору Go, що даний файл має бути скомпільований як **виконуваний файл** (binary), а не як бібліотека. Саме в цьому пакеті обов'язково має бути оголошена функція main().

3. Яке ім’я головного пакету?

Головний пакет завжди має ім’я main. Навіть якщо файл знаходиться у папці з іншою назвою, для створення точки входу в програму використовується саме package main.

4. Скільки файлів знаходиться у головному пакеті?

У головному пакеті може бути **будь-яка кількість файлів**. Важливо лише, щоб усі вони мали оголошення package main. Під час компіляції всі ці файли розглядаються як єдине ціле.

5. Де розміщуються пакети?

Зазвичай пакети розміщуються в окремих **директоріях**. Згідно з правилами Go, всі файли в одній директорії (за винятком тестових) повинні належати до одного пакета. Назва директорії, як правило, збігається з назвою пакета, що спрощує його імпортування.

6. Як організувати тестування пакетів?

Для тестування в Go використовується вбудований інструментарій:

Файли: Тести записуються у файли, назва яких закінчується на _test.go (наприклад, math_test.go).

Пакет: Використовується стандартна бібліотека testing.

Функції: Тестові функції повинні починатися з префікса Test (наприклад, func TestCalculate(t *testing.T)).

Запуск: Тести виконуються в терміналі за допомогою команди go test або go test -v для детального звіту.